

COMP3242/6242: Deep Learning

Week 6 — Sequence Models

Stephen Gould

`stephen.gould@anu.edu.au`

Australian National University

Semester 1, 2026

Overview

- ▶ what is a sequence model?
- ▶ recurrent neural networks
- ▶ back-propagation through time
- ▶ long short-term memory network
- ▶ sequence-to-sequence encoder-decoder models
- ▶ language models
 - ▶ encoding words as vectors
 - ▶ auto-regressive generation
- ▶ applications
 - ▶ image captioning
 - ▶ visual question answering
 - ▶ vision and language navigation

Reading and Resources.

- ▶ Andrej Karpathy, The Unreasonable Effectiveness of Recurrent Neural Networks, <https://karpathy.github.io/2015/05/21/rnn-effectiveness/>

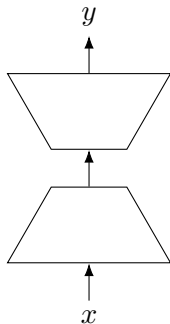
Sequence Inputs and Outputs

- ▶ unlike an image or a label, a sequence has **arbitrary length**

Sequence Inputs and Outputs

- ▶ unlike an image or a label, a sequence has **arbitrary length**

one-to-one

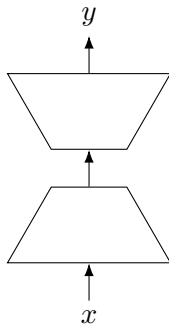


e.g., image classification

Sequence Inputs and Outputs

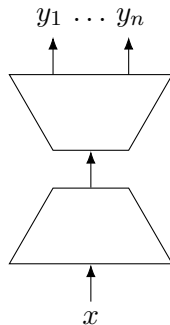
- ▶ unlike an image or a label, a sequence has **arbitrary length**

one-to-one



e.g., image classification

one-to-many

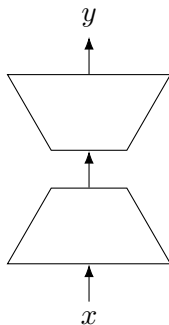


e.g., image captioning

Sequence Inputs and Outputs

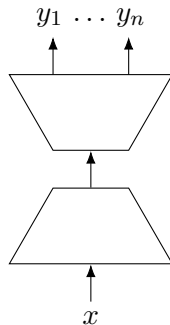
- ▶ unlike an image or a label, a sequence has **arbitrary length**

one-to-one



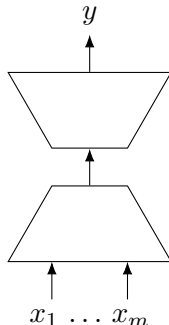
e.g., image classification

one-to-many



e.g., image captioning

many-to-one

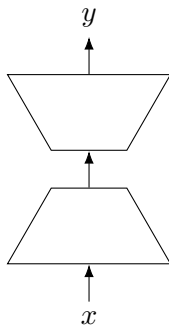


e.g., video classification

Sequence Inputs and Outputs

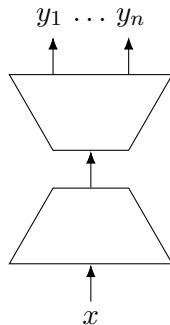
- ▶ unlike an image or a label, a sequence has **arbitrary length**

one-to-one



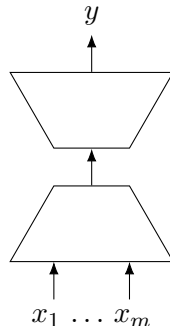
e.g., image classification

one-to-many



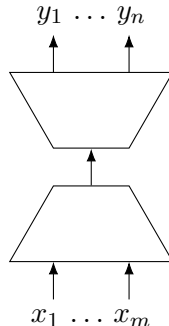
e.g., image captioning

many-to-one



e.g., video classification

many-to-many

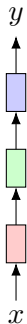


e.g., language
translation

Sequence Inputs and Outputs

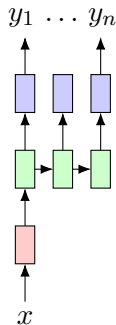
- ▶ unlike an image or a label, a sequence has **arbitrary length**

one-to-one



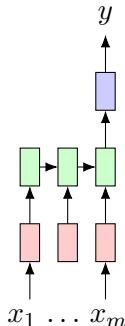
e.g., image classification

one-to-many



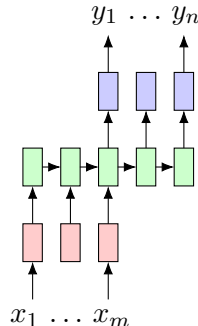
e.g., image captioning

many-to-one



e.g., video classification

many-to-many

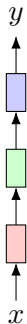


e.g., language
translation

Sequence Inputs and Outputs

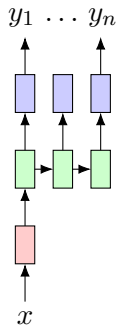
- ▶ unlike an image or a label, a sequence has **arbitrary length**

one-to-one



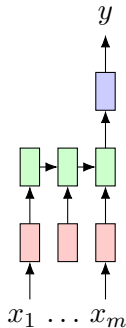
e.g., image classification

one-to-many



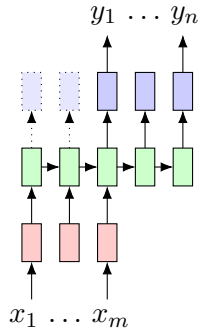
e.g., image captioning

many-to-one



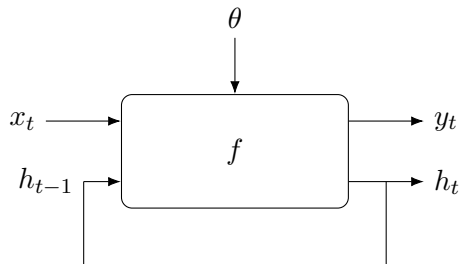
e.g., video classification

many-to-many



e.g., language
translation

Recurrent Neural Networks (RNNs)

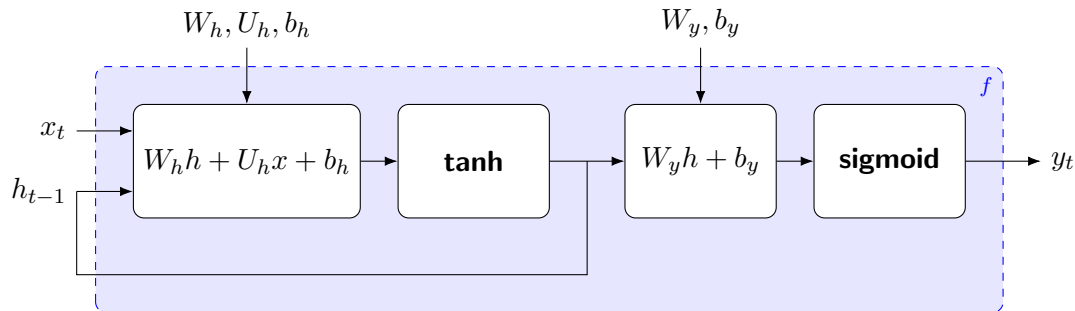


- feed the (partial) output of a network back into itself and time-step inference to give a recurrent network,

$$(y_t, h_t) = f(x_t, h_{t-1})$$

- same function (with same parameters) applied at each time-step
- h_t is called the **hidden** or **latent** state

Concrete Example



$$h_t = \mathbf{tanh}(W_h h_{t-1} + U_h x_t + b_h)$$

$$y_t = \mathbf{sigmoid}(W_y h_t + b_y)$$

Convolution as a (Linear) Recurrent Neural Network

- ▶ recall that a 1D convolution, $y = a * x$, can be written as

$$y_t^{\text{conv}} = \sum_{i=1}^p a_i x_{t+i-1}$$

- ▶ we can implement this as a recurrent neural network as follows
 - ▶ initialize $h_0 = (0, \dots, 0) \in \mathbb{R}^p$
 - ▶ define the RNN outputs as

$$\begin{aligned} h_t &= (h_{t-1}[2:p], x_t) \\ &= (x_{t-p+1}, \dots, x_{t-1}, x_t) \\ y_t^{\text{rnn}} &= a^T h_t \end{aligned}$$

- ▶ which gives the convolution output delayed by $p - 1$ time steps, i.e., $y_t^{\text{rnn}} = y_{t-p+1}^{\text{conv}}$

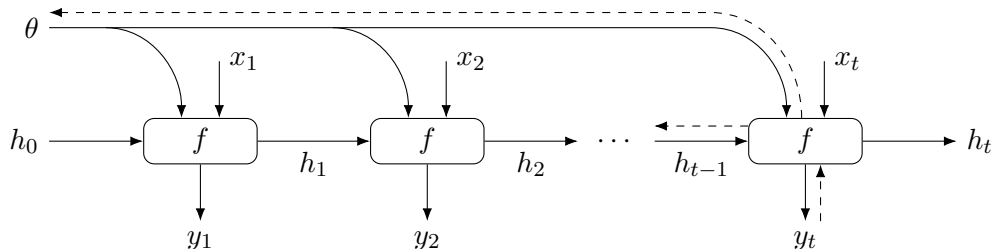
Back-propagation through Time

- ▶ train recurrent networks by **unrolling** the network to a given time horizon and back-propagating through time,

$$\begin{aligned}(y_t, h_t) &= f(x_t, h_{t-1}; \theta) \\ (y_{t-1}, h_{t-1}) &= f(x_{t-1}, h_{t-2}; \theta) \\ &\vdots \\ (y_1, h_1) &= f(x_1, h_0; \theta)\end{aligned}$$

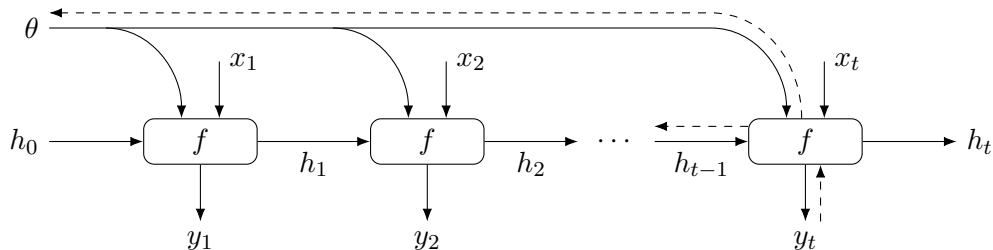
Back-propagation through Time

- ▶ train recurrent networks by **unrolling** the network to a given time horizon and back-propagating through time,



Back-propagation through Time

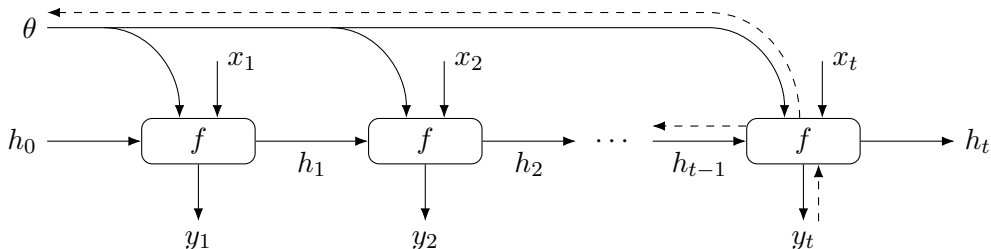
- ▶ train recurrent networks by **unrolling** the network to a given time horizon and back-propagating through time,



- ▶ note that the final output y_t depends on θ and h_{t-1} , which depends on θ and h_{t-2} , which depends on θ and $\dots$

Back-propagation through Time

- ▶ train recurrent networks by **unrolling** the network to a given time horizon and back-propagating through time,



- ▶ then (assuming the loss is applied to the last output, y_t)

$$\frac{\partial L}{\partial \theta} = \frac{\partial L}{\partial y_t} \frac{\partial y_t}{\partial \theta} + \frac{\partial L}{\partial y_t} \frac{\partial y_t}{\partial h_{t-1}} \frac{\partial h_{t-1}}{\partial \theta} + \dots + \frac{\partial L}{\partial y_t} \frac{\partial y_t}{\partial h_{t-1}} \left(\prod_{\tau=1}^{t-1} \frac{\partial h_{\tau+1}}{\partial h_\tau} \right) \frac{\partial h_1}{\partial \theta}$$

Vanishing and Exploding Gradients in RNNs

- ▶ gradient contribution from timestep $s < t - 1$ is

$$\frac{\partial L}{\partial y_t} \frac{\partial y_t}{\partial h_{t-1}} \left(\prod_{\tau=s}^{t-1} \frac{\partial h_{\tau+1}}{\partial h_{\tau}} \right) \frac{\partial h_s}{\partial \theta}$$

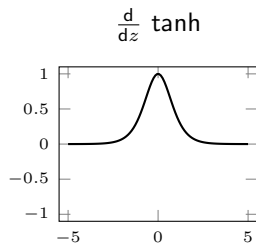
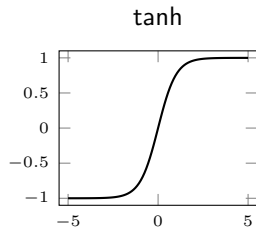
- ▶ consider $\frac{\partial h_{\tau+1}}{\partial h_{\tau}}$ for the **tanh** activation function,

$$h_t = \mathbf{tanh}(W_h h_{t-1} + U_h x_t + b_h)$$

- ▶ using $\frac{d}{dz} \tanh(z) = 1 - \tanh^2(z)$, we have

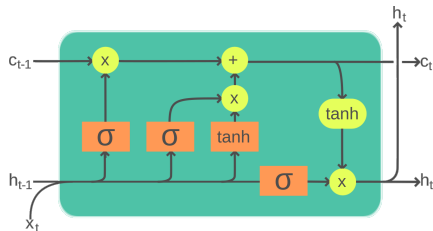
$$\frac{\partial h_{\tau+1}}{\partial h_{\tau}} = \mathbf{diag}(1 - h_{\tau+1}^2) W_h$$

- ▶ if any $h_{\tau+1}$ is close to ± 1 or $\sigma_{\max}(W_h) < 1$, then gradient vanishes
- ▶ if all $h_{\tau+1}$ are close to 0 and $\sigma_{\max}(W_h) > 1$, then gradient explodes



Long-short Term Memory (LSTMs)

(Hochreiter and Schmidhuber, 1997)



- popular RNN variant is the *long short-term memory* (LSTM) network

$$f_t = \sigma(W_f x_t + U_f h_{t-1} + b_f)$$

forget activations

$$i_t = \sigma(W_i x_t + U_i h_{t-1} + b_i)$$

input activations

$$o_t = \sigma(W_o x_t + U_o h_{t-1} + b_o)$$

output activations

$$c_t = f_t \circ c_{t-1} + i_t \circ \tanh(W_c x_t + U_c h_{t-1} + b_c)$$

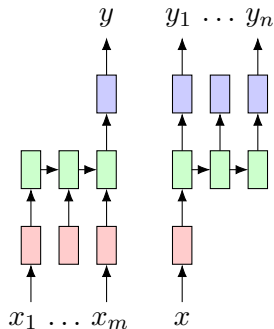
cell memory

$$h_t = o_t \circ \tanh(c_t)$$

cell output

- better propagation of information and gradients

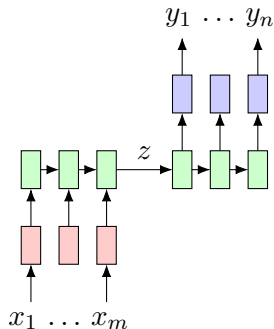
Sequence-to-sequence Encoder-decoder Model



seq-to-seq as **many-to-one** followed by **one-to-many**

- ▶ RNN encoder consumes arbitrary length input sequence
- ▶ final hidden state of the encoder is a summary of the input
- ▶ used to initialize the hidden state of the decoder
- ▶ RNN decoder produces arbitrary length output sequence

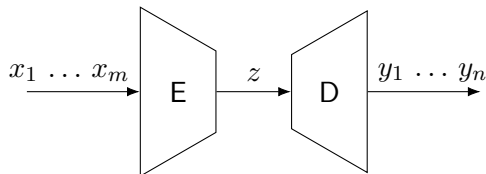
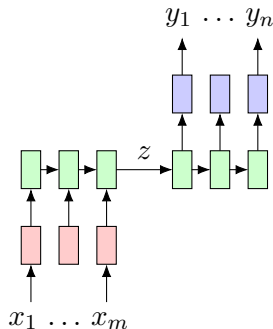
Sequence-to-sequence Encoder-decoder Model



seq-to-seq as **many-to-one** followed by **one-to-many**

- ▶ RNN encoder consumes arbitrary length input sequence
- ▶ final hidden state of the encoder is a summary of the input
- ▶ used to initialize the hidden state of the decoder
- ▶ RNN decoder produces arbitrary length output sequence

Sequence-to-sequence Encoder-decoder Model



seq-to-seq as **many-to-one** followed by **one-to-many**

- ▶ RNN encoder consumes arbitrary length input sequence
- ▶ final hidden state of the encoder is a summary of the input
- ▶ used to initialize the hidden state of the decoder
- ▶ RNN decoder produces arbitrary length output sequence

Language Models

- ▶ given a sequence of tokens (e.g., characters or words), predict the next token
- ▶ at each time step RNN outputs a probability distribution over all possible tokens

$$h_t = f(h_{t-1}, x_t)$$

$$y_t = \mathbf{softmax}(Ah_t + b)$$

- ▶ learn with cross-entropy loss on outputs
- ▶ target output at time t is the same as model input at time $t + 1$

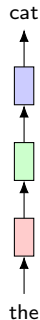
Language Models

- ▶ given a sequence of tokens (e.g., characters or words), predict the next token
- ▶ at each time step RNN outputs a probability distribution over all possible tokens

$$h_t = f(h_{t-1}, x_t)$$

$$y_t = \mathbf{softmax}(Ah_t + b)$$

- ▶ learn with cross-entropy loss on outputs
- ▶ target output at time t is the same as model input at time $t + 1$



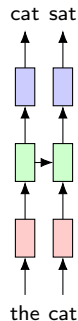
Language Models

- ▶ given a sequence of tokens (e.g., characters or words), predict the next token
- ▶ at each time step RNN outputs a probability distribution over all possible tokens

$$h_t = f(h_{t-1}, x_t)$$

$$y_t = \mathbf{softmax}(Ah_t + b)$$

- ▶ learn with cross-entropy loss on outputs
- ▶ target output at time t is the same as model input at time $t + 1$



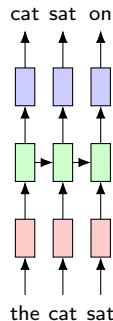
Language Models

- ▶ given a sequence of tokens (e.g., characters or words), predict the next token
- ▶ at each time step RNN outputs a probability distribution over all possible tokens

$$h_t = f(h_{t-1}, x_t)$$

$$y_t = \mathbf{softmax}(Ah_t + b)$$

- ▶ learn with cross-entropy loss on outputs
- ▶ target output at time t is the same as model input at time $t + 1$



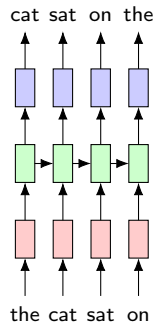
Language Models

- ▶ given a sequence of tokens (e.g., characters or words), predict the next token
- ▶ at each time step RNN outputs a probability distribution over all possible tokens

$$h_t = f(h_{t-1}, x_t)$$

$$y_t = \mathbf{softmax}(Ah_t + b)$$

- ▶ learn with cross-entropy loss on outputs
- ▶ target output at time t is the same as model input at time $t + 1$



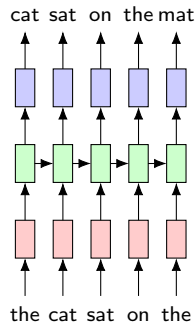
Language Models

- ▶ given a sequence of tokens (e.g., characters or words), predict the next token
- ▶ at each time step RNN outputs a probability distribution over all possible tokens

$$h_t = f(h_{t-1}, x_t)$$

$$y_t = \mathbf{softmax}(Ah_t + b)$$

- ▶ learn with cross-entropy loss on outputs
- ▶ target output at time t is the same as model input at time $t + 1$



Encoding Words as Vectors

- ▶ consider a vocabulary $\mathcal{V}$ of m words
- ▶ we can encode each word in $\mathcal{V}$ using one-hot encoding

$$\begin{array}{lll} \text{"dog"} & \rightarrow & e_1 \\ \text{"cat"} & \rightarrow & e_2 \\ \text{"duck"} & \rightarrow & e_3 \\ & \vdots & \end{array}$$

i.e., the i -th word in the list maps to $e_i = (0, \dots, 1, 0, \dots) \in \mathbb{R}^m$

Encoding Words as Vectors

- ▶ consider a vocabulary $\mathcal{V}$ of m words
- ▶ we can encode each word in $\mathcal{V}$ using one-hot encoding

$$\begin{array}{ccccccc} \text{"dog"} & \rightarrow & e_1 & \rightarrow & \boxed{Ax + b} & \rightarrow & z_1 \\ \text{"cat"} & \rightarrow & e_2 & \rightarrow & \boxed{Ax + b} & \rightarrow & z_2 \\ \text{"duck"} & \rightarrow & e_3 & \rightarrow & \boxed{Ax + b} & \rightarrow & z_3 \\ & & \vdots & & \vdots & & \end{array}$$

i.e., the i -th word in the list maps to $e_i = (0, \dots, 1, 0, \dots) \in \mathbb{R}^m$

- ▶ the first layer of an RNN is typically a linear layer, $z = Ax + b$, with $A \in \mathbb{R}^{n \times m}$
- ▶ ignoring b , the i -th word mapping becomes

$$z_i = Ae_i = a_i$$

where a_i is the i -th column of A

- ▶ so we can just directly learn **word embeddings**, i -th word to $z_i \in \mathbb{R}^n$

Training Language Models

- ▶ teacher-forcing approach to training, **next-token prediction**
 - ▶ instead of using the predicted word, feed in the ground-truth word
- ▶ minimize cross-entropy loss

$$L(\theta) = - \sum_{t=1}^T \log [\mathbf{softmax}(Af(h_{t-1}, x_t) + b)]_{w_t}$$

where w_t is the index of the correct ground-truth word

- ▶ we can chunk long sequences into segments of length T , by caching h_s for some $s > 0$ and treating as a constant

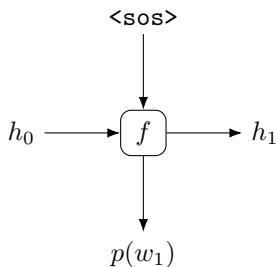
$$h_s = f \circ \dots \circ f \circ f(h_0, x_1, x_2, \dots, x_s)$$

$$L(\theta) = - \sum_{t=s+1}^{s+T} \log [\mathbf{softmax}(Af(h_{t-1}, x_t) + b)]_{w_t}$$

- ▶ word embeddings usually pre-trained separately (next lecture)

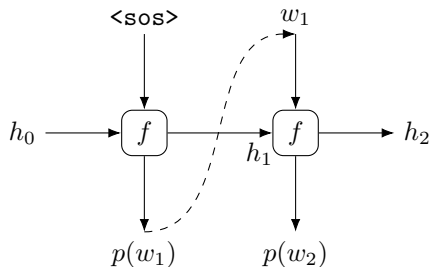
Auto-regressive Generation

- ▶ recurrent network f outputs a distribution over the next word, $p(w_t \mid h_t, w_{t-1})$
- ▶ model samples a word from the distribution, $w_t \sim p(w_t \mid h_t, w_{t-1})$
- ▶ the sampled word (and hidden state) provided as input for next time step
- ▶ start-of-sequence and end-of-sequence denoted by special tokens



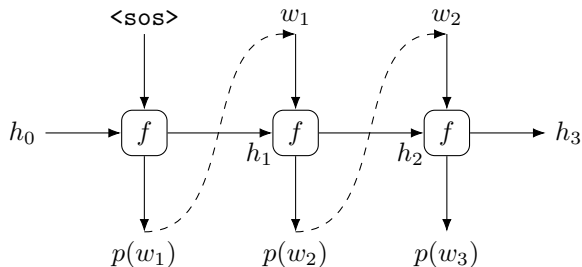
Auto-regressive Generation

- ▶ recurrent network f outputs a distribution over the next word, $p(w_t \mid h_t, w_{t-1})$
- ▶ model samples a word from the distribution, $w_t \sim p(w_t \mid h_t, w_{t-1})$
- ▶ the sampled word (and hidden state) provided as input for next time step
- ▶ start-of-sequence and end-of-sequence denoted by special tokens



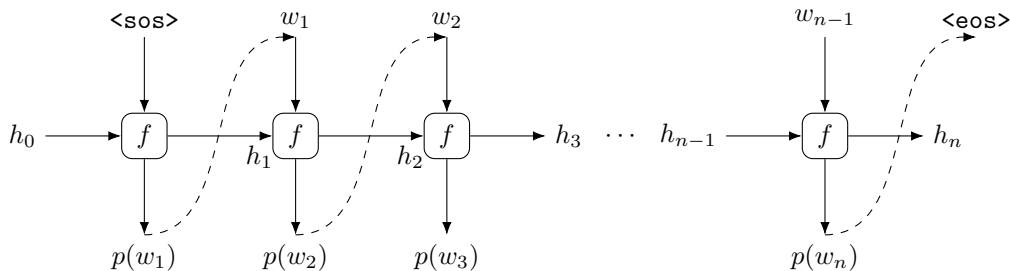
Auto-regressive Generation

- ▶ recurrent network f outputs a distribution over the next word, $p(w_t \mid h_t, w_{t-1})$
- ▶ model samples a word from the distribution, $w_t \sim p(w_t \mid h_t, w_{t-1})$
- ▶ the sampled word (and hidden state) provided as input for next time step
- ▶ start-of-sequence and end-of-sequence denoted by special tokens



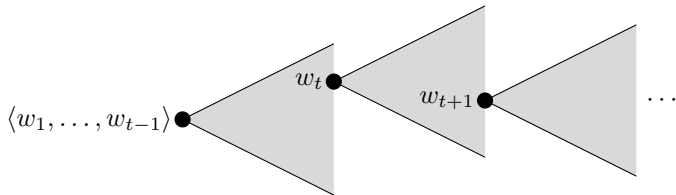
Auto-regressive Generation

- ▶ recurrent network f outputs a distribution over the next word, $p(w_t \mid h_t, w_{t-1})$
- ▶ model samples a word from the distribution, $w_t \sim p(w_t \mid h_t, w_{t-1})$
- ▶ the sampled word (and hidden state) provided as input for next time step
- ▶ start-of-sequence and end-of-sequence denoted by special tokens



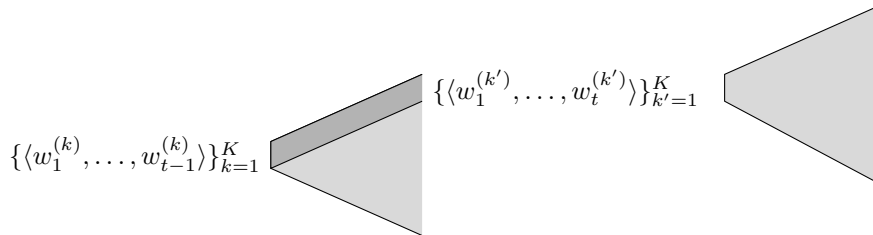
Beam Search

- ▶ auto-regressive generation is greedy
 - ▶ may not sample the highest likelihood sequence even if the most likely word is sampled at each time step, $w_t \in \arg \max_w p(w \mid h_t, w_{t-1})$
- ▶ to mitigate this, beam search maintains a set of partially generated sequences



Beam Search

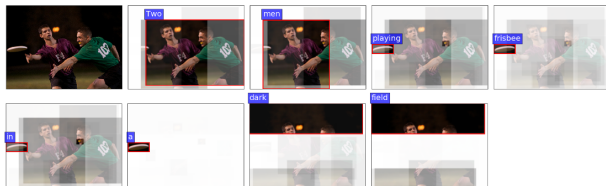
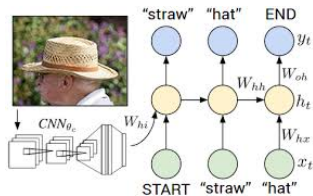
- ▶ auto-regressive generation is greedy
 - ▶ may not sample the highest likelihood sequence even if the most likely word is sampled at each time step, $w_t \in \arg \max_w p(w \mid h_t, w_{t-1})$
- ▶ to mitigate this, beam search maintains a set of partially generated sequences



- ▶ partial/full sequences scored by joint (log) probability, $\sum_{i=1}^t \log p(w_i \mid h_i, w_{i-1})$
 - ▶ other score functions can be used too, e.g., to compensate for sequence length
- ▶ **extension:** constrained beam search (Anderson et al., 2017)

Image Captioning

(Karpathy et al., 2015; Anderson et al., 2018)



- ▶ use CNN to extract feature vector(s) from image
- ▶ use RNN with beam search to output caption
 - ▶ image feature either initializes RNN hidden state, h_0 , or
 - ▶ concatenated with previous samples word as input at each time step, x_t
- ▶ difficult to evaluate since images can be described in numerous ways
 - ▶ n-gram based: BLEU, ROUGE, METEOR, CIDEr
 - ▶ scene graph based: SPICE (Anderson et al., 2016)

Image Captioning — Bottom-up Top-down Attention

(Anderson et al., 2018)

- ▶ idea: attend to different parts of the image as caption is generated
- ▶ set of attention candidates, $V = \{v_1, \dots, v_n\}$, comes from:
 - ▶ standard CNN feature map ($m \times m$ grid)
 - ▶ object-based detections (k regions), e.g., from Faster R-CNN

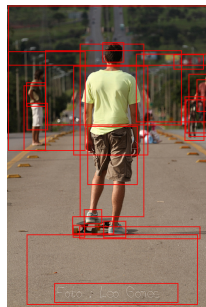
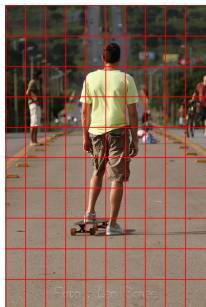


Image Captioning — Bottom-up Top-down Attention

(Anderson et al., 2018)

- ▶ idea: attend to different parts of the image as caption is generated
- ▶ set of attention candidates, $V = \{v_1, \dots, v_n\}$, comes from:
 - ▶ standard CNN feature map ($m \times m$ grid)
 - ▶ object-based detections (k regions), e.g., from Faster R-CNN
- ▶ combine bottom-up process (extracted salient regions, V) with top-down process (weighted based on context, h_t)

$$\hat{v}_t = g(h_{t-1}; V, \theta)$$

$$x_t = (w_{t-1}, \hat{v}_t)$$

$$p(w), h_t = f(h_{t-1}, x_t, \theta)$$

$$w_t \sim p(w)$$

Visual Question Answering

(Antol et al., 2015)

- ▶ task of answering questions about a given image (or video)



What color are her eyes?
What is the mustache made of?



How many slices of pizza are there?
Is this a vegetarian pizza?



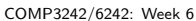
Is this person expecting company?
What is just under the tree?



Does it appear to be rainy?
Does this person have 20/20 vision?

- ▶ encode image using pre-trained CNN; encode question using pre-trained RNN
- ▶ combine into latent representation of the query image-question pair
- ▶ output can be open-ended (language model decoder) or multiple-choice (MLP with softmax decoder)

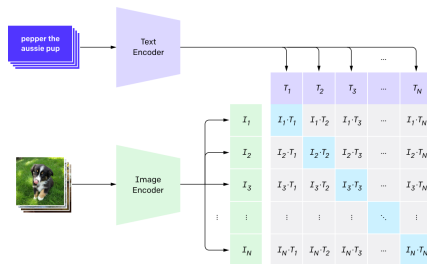
(Anderson et al., 2018; Hong et al., 2023)



Joint Vision and Language Embedding

(Radford et al., 2021; Li et al., 2022)

- ▶ learn visual representations aligned with language representations
 - ▶ i.e., embeds images and words/sentences in the same latent space
- ▶ training set of (image, text)-pairs, $\mathcal{D} = \{(I_i, T_i)\}_{i=1}^N$
- ▶ **contrastive loss**, $\ell_{i,j} = -\log \frac{\exp \phi(I_i, T_j)}{\sum_{k \neq i} \exp \phi(I_i, T_k)}$ where ϕ measures similarity
 - ▶ brings embeddings of I_i and T_j close for $i = j$ and far for $i \neq j$



- ▶ facilitates zero-shot tasks (e.g., prediction, retrieval)